RetailMind

Multi-Constraint Product Search with Hybrid Retrieval, Knowledge-Graph Filtering, and a Grounded Agent

An Information Retrieval project evaluated on Amazon ESCI

Abstract. RetailMind answers natural-language shopping queries that carry multiple constraints (brand, colour, material, price, delivery, stock). It combines sparse and dense retrieval, reciprocal rank fusion, cross-encoder re-ranking, a knowledge-graph constraint filter, and a LangGraph agent that self-checks results and writes a grounded recommendation with a local LLM. Evaluated against a BM25 baseline on Amazon ESCI via an A0–A6 ablation on held-out test queries, the retrieval stack lifts nDCG@10 from 0.470 to 0.575 and MRR@10 from 0.601 to 0.715. On a synthetic constrained set (SC-100) the knowledge-graph filter raises nDCG@10 from 0.346 to 0.658 and cuts top-10 constraint violations from ~57% to 3%. The grounded generator cites only products it was shown (citation validity 1.00). Limitations of the synthetic evaluation are stated openly.

1. Introduction and Research Questions

Keyword search handles “running shoes” but struggles with “red Nike running shoes under $80 delivered in two days”: the request mixes a product need, soft attributes, and hard constraints. RetailMind separates the two problems — a retrieval stack matches intent, and a constraint layer enforces requirements over a product knowledge graph — then an agent checks the results, retries if weak, and writes a recommendation grounded strictly in retrieved products.

- **RQ1.** Does hybrid retrieval (BM25 + dense) with RRF beat a BM25 baseline on ESCI?
- **RQ2.** Does cross-encoder re-ranking improve ranking over fusion?
- **RQ3.** Can a parsed-constraint filter enforce hard requirements without losing recall?
- **RQ4.** Does the agent + grounded generation layer produce faithful answers?
- **RQ5.** What does Amazon ESCI let us demonstrate, and what does it not?

2. Dataset

2.1 Amazon ESCI (Shopping Queries)

We use the public ESCI dataset (Task 1, US locale, small version): an examples table of query–product judgements and a products table of text. Each judgement carries a label with a graded relevance gain (scaled by 100 so Substitute/Complement signal survives integer storage in ranx).

Label	Meaning	Graded gain
E — Exact	the product satisfies the query	1.0
S — Substitute	a reasonable alternative	0.1
C — Complement	used together with the item	0.01
I — Irrelevant	does not fit	0.0

2.2 Prepared corpus

From the US small set (29,844 queries) we sample **300 development** queries (from the ESCI train split) and **700 test** queries (from the ESCI test split), keeping tuning separate from held-out evaluation. The corpus holds every judged product plus 10,000 distractors (**29,775 products**). Product text is cleaned and the title, brand, colour, bullets and description are concatenated (capped at 2,000 chars).

2.3 SC-100: a synthetic constrained set

ESCI queries rarely state price or delivery limits, so they cannot test constraint compliance. SC-100 adds 100 dev + 100 test queries built from ESCI queries with appended constraint phrases (“under $40”, “delivered in 2 days”, “in stock”). Each product receives **synthetic** price/delivery/stock values; a product is relevant only if ESCI judged it relevant *and* it meets the constraints. Limits are drawn so some Exact products pass and some fail. These values are synthetic and labelled as such throughout.

3. System Architecture

The pipeline is a sequence of independently switchable stages:

- **Query understanding.** A deterministic rule/lexicon parser extracts constraints (brand/colour learned from the catalogue); an LLM decomposes multi-need queries.
- **Retrieval.** Okapi BM25 with the Lucene idf, plus all-MiniLM-L6-v2 dense embeddings in an exact FAISS index, fused by reciprocal rank fusion (k=60, rank-only, no score scaling).
- **Re-ranking.** ms-marco-MiniLM-L-6-v2 cross-encoder re-scores the top 50; the tail keeps its order, so recall at depth is never reduced.
- **Constraint filter.** A NetworkX knowledge graph links products to brand/colour/material and holds price/delivery/stock. Strict mode removes violators; lenient mode demotes them (large penalty for hard limits sinks them below compliant items without losing recall).
- **Agent & generation.** A LangGraph loop self-checks, relaxes and retries (≤2 extra attempts), then a grounded generator writes an answer citing only shown products. The LLM is any OpenAI-compatible endpoint; this project used qwen2.5 via Ollama.

4. Experimental Setup

- **Metrics.** nDCG@10 (graded), nDCG@10 (Exact), MRR@10, Recall@100 via ranx. SC-100 adds violation@10 and clean@10. Answers: citation validity, grounded rate, abstention, relevancy.
- **Splits.** All tuning on dev; every headline number on the untouched test split. Paired bootstrap intervals (5,000 resamples) judge significance.
- **Ladder.** A0 BM25 · A1 dense · A2 BM25+dense (RRF) · A3 +re-rank · A4 +parse+filter · A5 +decomposition · A6 +agent+grounded generation.
- **Engineering.** src-layout package, strict Pydantic config, 167 passing tests, ruff-clean; LLM responses cached for reproducibility.

5. Results

5.1 Retrieval ablation on ESCI (test, 700 queries)

Run	Components	nDCG@10	nDCG@10 (E)	MRR@10	Recall@100
A0	BM25	0.4705	0.4591	0.6008	0.7727
A1	dense	0.4684	0.4579	0.6101	0.8000
A2	BM25 + dense (RRF)	0.5074	0.4961	0.6418	0.8401
A3	A2 + re-rank	0.5746	0.5631	0.7139	0.8401
A4	A3 + parse + filter	0.5748	0.5634	0.7151	0.8401

- **RQ1 — fusion helps.** A2 beats A0 on every metric (nDCG +0.037, MRR +0.041, Recall +0.067). Dense alone ties BM25, but the two miss different products, so fusion recovers recall (0.773→0.840).

- **RQ2 — re-ranking helps most.** A3 adds +0.067 nDCG and +0.072 MRR over A2 (intervals exclude zero); recall unchanged by design. Over the baseline, A3 improves nDCG by +0.104.
- **A4 neutral on ESCI (+0.0002).** ESCI queries carry no explicit limits, and brand/colour are weak discriminators here, so the filter is kept lenient and proven on SC-100.

5.2 Constraint evaluation on SC-100 (test, 100 queries)

Run	nDCG@10	MRR@10	Recall@100	violation@10	clean@10
A3 (no filter)	0.3457	0.3892	0.8006	~0.57 *	~0.02 *
A4 (+ filter)	0.6575	0.7541	0.8424	0.0300	0.9500
A6 (full system)	0.6479	0.7305	0.8347	0.0300	0.9500

* no-filter violation/clean measured on dev (A3 does not load the graph on the test run); representative of the unfiltered system.

- **RQ3 — the filter enforces constraints without losing recall.** It almost doubles nDCG@10 (0.346→0.658), slightly raises recall, and cuts top-10 violations from ~57% to 3%; 95% of queries have a fully clean top 10. Recall is preserved because hard-limit violators are demoted, not deleted.
- **A4 ≈ A6 on ranking.** Decomposition and the agent leave the ordering essentially unchanged; their contribution is the grounded answer.

5.3 Answer quality (A6, SC-100 test)

Metric	Value	Reading
citation_validity	1.0000	every cited product was shown — no invented products
grounded_rate	0.9865	98.7% of answers cite only valid products
mean_citations	1.47	one or two products cited per answer
abstention_rate	0.2600	declines ~26% rather than overclaim
answer_relevancy	0.7098	answers on-topic with the query (cosine)

RQ4 — answers are faithful. Across 100 test queries the generator never cited a product outside the retrieved set. The ~26% abstention reflects honest behaviour when nothing clearly matches.

6. Discussion

The ablation decomposes cleanly: **fusion fixes recall** (brings in products either retriever misses), **re-ranking fixes ordering** (moves Exact matches to the top), **the filter fixes constraint compliance** (invisible on ESCI, decisive on SC-100), and **the agent adds a faithful answer** without disturbing the ranking. Each stage earns its place for a separable, measured reason rather than hiding behind one aggregate number.

7. Limitations

- **SC-100 is partly circular** — relevance is defined by the same constraints the filter enforces, and the parser sees phrasings we generated. It shows the mechanism works end to end, not that arbitrary real phrasing parses as cleanly.
- **Price/delivery/stock are synthetic** — ESCI has none; these fields exist only to exercise the constraint layer and are labelled synthetic.
- **Answer quality is measured on SC-100** — a real-query answer evaluation would strengthen the claim.
- **Not comparable to ESCI leaderboards** — we use a 29,775-product subset and our own sampled queries.
- **Small local LLM** (qwen2.5) — a larger model would likely abstain less; the grounding contract is unchanged.

8. Conclusion and Future Work

RetailMind takes product search from a BM25 baseline (nDCG@10 0.470) to a re-ranked hybrid system (0.575) on Amazon ESCI, and a knowledge-graph constraint layer enforces hard requirements — demonstrated on SC-100, where it lifts nDCG@10 from 0.346 to 0.658 and reduces violations from ~57% to 3% — topped by an agent producing faithful, fully grounded recommendations (citation validity 1.00). Future work: a human-labelled constrained set to remove SC-100's circularity; a learned constraint parser; real catalogue attributes; and a larger answer model with a real-query faithfulness study.

9. Reproducibility

Data → download_esci → prepare_data → build_lexicon → build_attributes → make_sc_set. Indexes → build_dense_index. Evaluation → run_retrieval (A0–A4), run_pipeline (A5–A6), build_report. Config in configs/default.yaml; LLM endpoint in .env (LLM_BASE_URL / LLM_MODEL / LLM_API_KEY). 167 unit tests, ruff-clean, cached LLM calls.